

- **Group 5**
- Student names:
 - Stella Kiarie
 - Dapash Kumali
 - Doris Mutie
 - Morvine Otieno
- Student pace **Remote full time**
- Scheduled project review date/time: **6th – 11th February, 2026**
- Instructor name: **Nikita Njoroge**

Movie Recommendation System

1. Business Understanding

Introduction

Overview: This project develops a personalized movie recommendation system to help a streaming platform improve user engagement and retention. Using the MovieLens Small dataset, we leverage explicit user ratings to provide direct feedback on user preferences, which is essential for supervised collaborative filtering.

Challenges: Users frequently get overwhelmed by the sheer volume of available content. Poor recommendations lead to “choice paralysis,” decreased watch time, and increased user churn.

Proposed Solution: The business requires a system to predict user enjoyment and provide a personalized Top-5 recommendation matrix. Our solution utilizes a hybrid strategy - combining Singular Value Decomposition (SVD) with content-based genre similarity - to ensure robust performance even for new movies.

Brief Conclusion: The envisioned outcome is a recommendation engine that converts historical interaction into accurate, real-time predictions, providing actionable insights for stakeholders.

Problem Statement

Movie streaming platforms face a significant challenge in maintaining user engagement due to the difficulty of recommending content users truly enjoy. With a vast library of titles, users frequently suffer from choice paralysis and discovery fatigue. When recommendations are poor, it results in decreased watch time, lower user satisfaction, and increased platform churn, ultimately threatening the platform's retention and long-term growth.

Objectives

1. **Predict Ratings:** Predict ratings for unseen movies for each user to project future behavior.
2. **Personalized Recommendations:** Generate a tailored top-5 movie recommendation list for all users.
3. **Model Evaluation:** Evaluate recommendation performance using RMSE for rating prediction and Precision@5 / Recall@5 for top-N recommendations.
4. **Hybrid Strategy:** Implement a hybrid approach combining collaborative filtering and content-based methods to address cold-start challenges.
5. **Stakeholder Insights:** Provide actionable insights for stakeholders to enhance user engagement and platform retention.

2. DATA UNDERSTANDING

To build this recommendation system, we leveraged the MovieLens Latest Datasets. The dataset contains data generated by 610 users between March 19, 1996 and September 24, 2018.

The data used in this project is generated from two separate csv files merged to form one dataset. The data is gathered from two basic files : movies.csv and ratings.csv

Movie Information (movies.csv)

This dataset contains movie information, including titles and genres.

Columns:

- **movieId:** Unique identifier for each movie.
- **title:** The title of the movie, which also includes the year of release in parentheses.
- **genres:** A pipe-separated list of genres to categorize the movie (e.g., Action|Adventure|Comedy).

Ratings Data (ratings.csv)

Each entry represents a user's rating for a specific movie.

Contains user ratings on a 5-star scale for movies. Columns:

- **userId:** ID representing the unique identifier for each user.
- **movieId:** Unique identifier for each movie.
- **rating:** User's rating for the movie on a 5-star scale with half-star increments (0.5 to 5.0).
- **timestamp:** The timestamp when the rating was recorded, represented in seconds since midnight Coordinated Universal Time (UTC) of January 1, 1970.

Data Understanding Steps

1. **Shape of Data:** Calculating the total number of rows and columns in the dataset. Using the 'Shape' This will be used to determine the scale the dataset and allow us to calculate Sparsity.
2. **Data Types and dataset information:** To check and view the datatypes of various columns
3. **Completeness of Dataset:** To identify missing values using the isnull() sum().
4. **Uniqueness:** Checking for duplicates within the dataset.
 - Ensures there are no duplicate entries for the same user-movie pair, which would skew the recommendation logic.
5. **Uniformity:** Checking for consistent format across rows. For the genres columns, genres are pipe-separated and hence converting the genres into lists for better processing.
6. **Consistency:** We verified that the MovieId in the ratings table exists in the movies table. Merging of columns ensure that each ratings is linked to a valid movie title.
7. **Descriptive Analysis:** Using the .describe() to check for mean, standard deviation and quartiles of ratings.
8. **Validity:** Verify that movie ratings fall between 0.5 and 5.0 ensuring there are no outlier values in ratings such as 100

3. Data Preparation

a) import the libraries

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

b) load the datasets

```
In [2]: # Load datasets (use Dataset/ folder)
from pathlib import Path
DATA_DIR = Path("Dataset")
ratings = pd.read_csv(DATA_DIR / "ratings.csv")
movies = pd.read_csv(DATA_DIR / "movies.csv")
print(ratings.head(),
      movies.head())
```

```
In [3]: ## Checking number of rows and columns in the dataset
shape_summary = pd.DataFrame({
    'Dataset': ['Movies Dataset', 'Ratings Dataset'],
    'Total Rows': [movies.shape[0], ratings.shape[0]],
    'Total Columns': [movies.shape[1], ratings.shape[1]],
    'Unique IDs': [len(movies['movieId'].nunique()), ratings['userId'].nunique()]
}).reset_index('Dataset')
```

```
In [4]: ## Viewing shape of the dataframes
print(f"--- DATASET DIMENSIONS ---")
display(shape_summary)

--- DATASET DIMENSIONS ---
```

	Total Rows	Total Columns	Unique IDs
Dataset			
Movies Dataset	9742	3	9742
Ratings Dataset	100836	4	610

```
In [5]: # A view of the dataset info
print(f"--- MOVIES INFO ---")
movies.info()

print(f"--- RATINGS INT INFO ---")
ratings.info()
```

```
--- MOVIES INFO ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9742 entries, 0 to 9741
Data columns (total 3 columns):
 # Column Non-Null Count Dtype
---
 0 movieId 9742 non-null int64
 1 title 9742 non-null object
 2 genres 9742 non-null object
dtypes: int64(1), object(2)
memory usage: 228.5+ KB
```

```
--- RATINGS INT INFO ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100836 entries, 0 to 100835
Data columns (total 4 columns):
 # Column Non-Null Count Dtype
---
 0 userId 100836 non-null int64
 1 movieId 100836 non-null int64
 2 rating 100836 non-null float64
 3 timestamp 100836 non-null int64
dtypes: float64(1), int64(3)
memory usage: 3.1 MB
```

```
In [6]: ## Checking for missing values
missing_values = pd.DataFrame({
    'Movies': movies.isnull().sum(),
    'Ratings': ratings.isnull().sum()
}).fillna(0)
display(missing_values)
```

	Movies	Ratings
genres	0	N/A
movieId	0	0
rating	N/A	0
timestamp	N/A	0
title	0	N/A
userId	N/A	0

```
In [7]: ## Checking for Descriptive Analysis
# Focus on the 'rating' column distribution
display(ratings[['rating']].describe().T)
```

	count	mean	std	min	25%	50%	75%	max
rating	100836.0	3.501557	1.042529	0.5	3.0	3.5	4.0	5.0

Data Sparsity

- Users: `n_users = ratings['userId'].nunique()`
- Movies: `n_movies = ratings['movieId'].nunique()`
- Sparsity: `1 - (n_ratings / (n_users * n_movies))`

High sparsity motivates matrix factorization and hybrid strategies.

```
In [8]: plt.style.use('ggplot')

## Calculating SPARSITY
n_users = ratings['userId'].nunique()
n_movies = ratings['movieId'].nunique()
possible_ratings = n_users * n_movies
actual_ratings = len(ratings)

sparsity = (1 - (actual_ratings / possible_ratings)) * 100
print(f"Matrix Sparsity: {sparsity:.4f}%")
```

Matrix Sparsity: 98.3000%

Interpretation

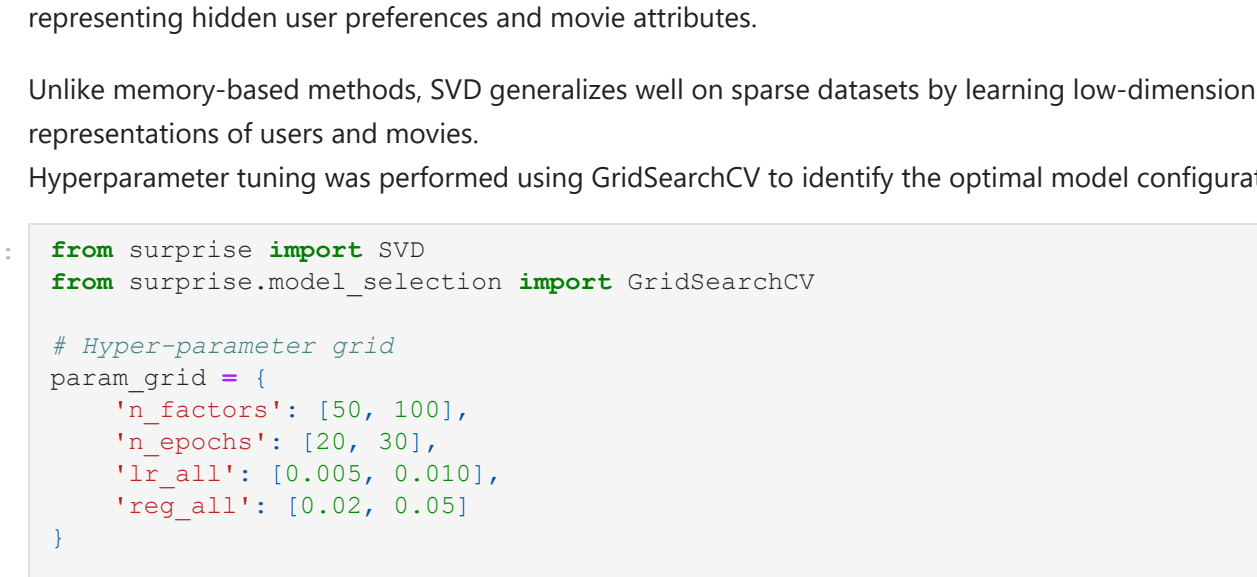
The user-item rating matrix is highly sparse, with a sparsity level of approximately 98.3%. This means that the vast majority of possible user-movie interactions are unobserved, which is typical in real-world recommender systems where users rate only a small subset of available items. High sparsity motivates the use of matrix factorization techniques such as SVD, which are well-suited for uncovering latent patterns in sparse data, and also highlights the importance of incorporating a hybrid strategy to mitigate cold-start scenarios.

EXPLORATORY DATA ANALYSIS (EDA)

```
In [9]: # VISUALIZING RATINGS DISTRIBUTION AND RATINGS PER MOVIE
fig, ax = plt.subplots(1, 2, figsize=(16, 6))

# Checking for the distribution of movie ratings
sns.countplot(x='rating', data=ratings, palette='viridis', ax=ax[0])
ax[0].set_title('Frequency of Rating Scores', fontsize=14)
ax[0].set_xlabel('Rating Score')
ax[0].set_ylabel('Count')
```

```
## PLOT FOR RATINGS PER MOVIE
movie_popularity = ratings.groupby('movieId').size().sort_values(ascending=False)
ax[1].plot(movie_popularity.values, color='teal', linewidth=3)
ax[1].fill_between(range(len(movie_popularity)), movie_popularity.values, color='teal')
ax[1].set_title('Ratings per Movie', fontsize=14)
ax[1].set_xlabel('Movies (Ranked by Popularity)')
ax[1].set_ylabel('Number of Ratings Received')
```



```
In [10]: print(f"Mean Global Rating: {round(ratings['rating'].mean(), 2)}")

Mean global rating: 3.5.
```

```
In [11]: mean_ratings = ratings.groupby('userId')['rating'].mean()
print(f"Mean rating per user: {round(mean_ratings.mean(), 2)}")

Mean ratings per user: 3.66.
```

Most ratings fall between 3 and 4, suggesting users are more likely to rate movies they expect to enjoy. Collaborative filtering is suitable to infer preferences from sparse behavior.

Which the the top and least rated movies?

```
In [12]: ## Evaluating the least rated movies
lowest_ratings = ratings.groupby('movieId')['rating'].mean()
movies[movies['movieId'] == lowest_ratings.idxmin()]

Out[12]:
```

movieId	title	genres
2689	3604 Gypsy (1962)	Musical

```
In [13]: ## Evaluating the Top Rated Movies
highest_rated = mean_ratings['rating'].idxmax()
movies[movies['movieId'] == highest_rated]

Out[13]:
```

movieId	title	genres
48	53 Lamerica (1994)	Adventure Drama

```
In [14]: ## Merging the Two Datasets
data = ratings.merge(movies, on='movieId')
data.head()
```

	userId	movieId	rating	timestamp	title	genres
0	1	1	4.0	964982703	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
1	5	1	4.0	847434962	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
2	7	1	4.5	1106635946	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
3	15	1	2.5	1510577970	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
4	17	1	4.5	1305696483	Toy Story (1995)	Adventure Animation Children Comedy Fantasy

A look at the merged dataframe indicates that some data cleaning is required in our datasets .

1. The Genre column is a string separated by a pipe . We can separate this column to a list for better analysis

```
In [15]: data['genres'] = data['genres'].apply(lambda x: x.split("|"))
data.head()
```

	userId	movieId	rating	timestamp	title	genres
0	1	1	4.0	964982703	Toy Story (1995)	[Adventure, Animation, Children, Comedy, Fantasy]
1	5	1	4.0	847434962	Toy Story (1995)	[Adventure, Animation, Children, Comedy, Fantasy]
2	7	1	4.5	1106635946	Toy Story (1995)	[Adventure, Animation, Children, Comedy, Fantasy]
3	15	1	2.5	1510577970	Toy Story (1995)	[Adventure, Animation, Children, Comedy, Fantasy]
4	17	1	4.5	1305696483	Toy Story (1995)	[Adventure, Animation, Children, Comedy, Fantasy]

4. Data Modelling

Model Selection

To build the movie recommendation system, we used a **collaborative filtering approach based on matrix factorization**, specifically the **Singular Value Decomposition (SVD)** algorithm.

Why Collaborative Filtering?

Collaborative filtering (CF) leverages **historical user-item interactions** to predict future user preferences. It is well suited for this project because:

- **Data sparsity:** Most users rate only a small fraction of available movies. CF learns patterns from observed ratings to make predictions.
- **Minimal reliance on content features:** CF does not require extensive movie metadata, making it effective even when item features are limited.
- **Personalization:** CF captures latent user preferences by identifying similarities between users or items.
- **Proven effectiveness:** CF is widely used in real-world systems such as Netflix and Amazon.

Collaborative Filtering Techniques Used

This project explores two collaborative filtering approaches:

1. **Item-Item Collaborative Filtering:**
 - Recommends movies that are similar to those a user has previously rated highly, based on item similarity.
2. **Matrix Factorization (SVD):**
 - Leans latent factors that represent hidden user preferences and movie characteristics, enabling accurate rating predictions even with sparse data.

A hybrid strategy is conceptually considered to address **cold-start scenarios**, where new users or new movies lack sufficient interaction history.

```
In [18]: from surprise import Dataset, Reader

reader = Reader(rating_scale=(0.5, 5))
dataset = Dataset.load_from_df(data[['userId', 'movieId', 'rating']], reader)
```

Baseline Model

The baseline model predicts ratings using global and user-level averages. This provides a reference point for evaluating the performance of more advanced collaborative filtering models.

```
In [19]: from surprise import BaselineOnly
from surprise.model_selection import cross_validate

baseline_model = BaselineOnly()
baseline_results = cross_validate(baseline_model, dataset, measures=['RMSE'], cv=5, verbose=False)
np.mean(baseline_results['test_rmse'])

Estimating biases using als...
Estimating biases using als...
Estimating biases using als...
Estimating biases using als...
0.872780336594638
```

Interpretation

The ALS-based collaborative filtering model achieved an RMSE of **0.87**, indicating strong predictive accuracy on a highly sparse user-item rating matrix. This suggests that the model effectively captures latent user preferences and movie characteristics while accounting for systematic rating biases.

Item-Item Collaborative Filtering (kNN)

Item-item collaborative filtering recommends movies that are similar to those a user has previously rated highly.

Cosine similarity was used to compute similarity between movies based on user ratings.

```
In [20]: from surprise import KNNBasic

sim_options = {'name': 'cosine', 'user_based': False}
knn_model = KNNBasic(sim_options=sim_options)
knn_results = cross_validate(knn_model, dataset, measures=['RMSE'], cv=5, verbose=False)
np.mean(knn_results['test_rmse'])

Computing the cosine similarity matrix...
Done computing similarity matrix...
Computing the cosine similarity matrix...
Done computing similarity matrix...
Computing the cosine similarity matrix...
Done computing similarity matrix...
Computing the cosine similarity matrix...
Done computing similarity matrix...
0.976472943740486
```

Interpretation

The item-item collaborative filtering model using cosine similarity achieved an RMSE of **0.98**, indicating lower predictive accuracy compared to the matrix factorization approach. This suggests that similarity-based methods struggle to fully capture user preferences in highly sparse rating data.

Matrix Factorization Using SVD with Hyperparameter Tuning

Singular Value Decomposition (SVD) is a matrix factorization technique that captures **latent factors** representing hidden user preferences and movie attributes.

Unlike memory-based methods, SVD generalizes well on sparse datasets by learning low-dimensional representations of users and movies.

Hyperparameter tuning was performed using GridSearchCV to identify the optimal model configuration.

```
In [21]: from surprise import SVD
from surprise.model_selection import GridSearchCV

# Hyperparameter grid
param_grid = {
    'n_factors': [50, 100],
    'n_epochs': [20, 30],
    'lr_all': [0.005, 0.01],
    'reg_all': [0.02, 0.05]
}

gs = GridSearchCV(SVD, param_grid, measures=['rmse'], cv=3)
gs.fit(dataset)

# Best parameters and RMSE
gs.best_params_['rmse'], gs.best_score_['rmse']

Out[21]:
```

```
((('n_factors': 100, 'n_epochs': 30, 'lr_all': 0.01, 'reg_all': 0.05),
0.8668007599179007))
```

Interpretation

The SVD model, tuned with 100 latent factors, 30 epochs, learning rate 0.01, and regularization 0.05, achieved an RMSE of **0.867**. This demonstrates that hyper-parameter tuning significantly improved predictive accuracy over baseline and item-item models.

```
In [22]: # Train SVD with best parameters
best_svd = SVD(**gs.best_params_['rmse'], random_state=42)
from surprise.model_selection import train_test_split

trainset, testset = train_test_split(dataset, test_size=0.2, random_state=42)
best_svd.fit(trainset)
predictions = best_svd.test(testset)
```

Top-5 Recommendations for User 1:

High sparsity motivates matrix factorization and hybrid strategies.

```
plt.style.use('ggplot')

## Calculating SPARSITY
n_users = ratings['userId'].nunique()
n_movies = ratings['movieId'].nunique()
possible_ratings = n_users * n_movies
actual_ratings = len(ratings)

sparsity = (1 - (actual_ratings / possible_ratings)) * 100
print(f"Matrix Sparsity: {sparsity:.4f}%")
```

Matrix Sparsity: 98.3000%

Interpretation

The user-item rating matrix is highly sparse, with a sparsity level of approximately 98.3%. This means that

Top-5 Recommendations for User 50:

```
# use of matrix factorization techniques such as SVD, which are well-suited for uncovering latent patterns in sparse data, and also highlights the importance of incorporating a hybrid strategy to mitigate cold-start scenarios.
```

EXPLORATORY DATA ANALYSIS (EDA)

```
# VISUALIZING RATINGS DISTRIBUTION AND RATINGS PER MOVIE

fig, ax = plt.subplots(1, 2, figsize=(16, 6))

# Checking for the distribution of movie ratings
sns.countplot(x='rating', data=ratings, palette='viridis', ax=ax[0])
ax[0].set_title('Frequency of Rating Scores', fontsize=14)
ax[0].set_xlabel('Rating Score')
ax[0].set_ylabel('Count')
```

Top-5 Recommendations for User 100:

```
ax[1].fill_between(range(len(movie_popularity)), movie_popularity.values, color='t
ax[1].set_title('Ratings per Movie', fontsize=14)
ax[1].set_xlabel('Movies (Ranked by Popularity)')
ax[1].set_ylabel('Number of Ratings Received')

plt.tight_layout()
plt.show()
```

The image displays two plots side-by-side. The left plot, titled "Frequency of Rating Scores", is a bar chart with "Count" on the y-axis (ranging from 0 to 20,000) and "Number of Ratings Received" on the x-axis (ranging from 0 to 5). The bars are colored in a gradient from light blue to dark blue. The right plot, titled "Ratings per Movie", is a line plot with "Number of Ratings Received" on the y-axis (ranging from 0 to 300) and "Movies (Ranked by Popularity)" on the x-axis (ranging from 0 to 10,000). The line is a solid dark blue.

Distribution of Predicted Ratings

To further understand the behavior of the hybrid recommendation system, the distribution of predicted ratings across Top-5 recommendations was visualized for a subset of users.

This analysis helps assess whether the model tends to recommend consistently high-rated movies.

```
In [28]: # Distribution of predicted ratings across all recommended movies
all_predicted = []

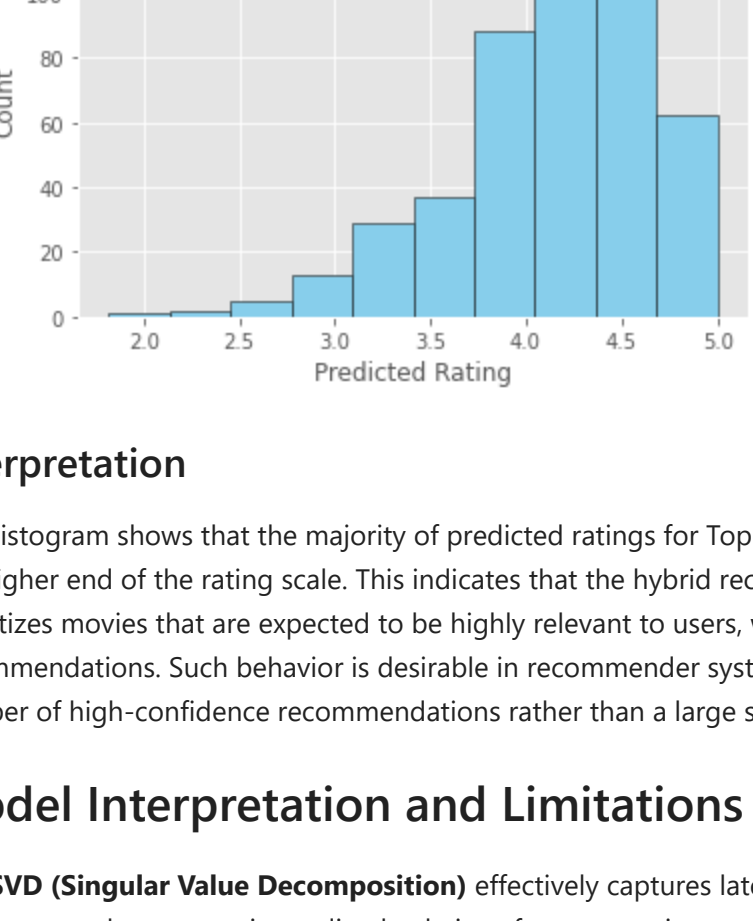
for user_id in ratings['userId'].unique()[1:100]:
    rec_df = hybrid_top_n_with_ratings(user_id)
    all_predicted.extend(rec_df['predicted_rating'].values)

plt.hist(all_predicted, bins=10, color='skyblue', edgecolor='black')
```



```
plt.title("Distribution of Predicted Ratings Across Top-5 Recommendations")
plt.xlabel("Predicted Rating")
plt.ylabel("Count")
plt.show()
```

Distribution of Predicted Ratings Across Top-5 Recommendations



Interpretation

The histogram shows that the majority of predicted ratings for Top-5 recommendations are concentrated at the higher end of the rating scale. This indicates that the hybrid recommendation system consistently prioritizes movies that are expected to be highly relevant to users, while filtering out low-quality recommendations. Such behavior is desirable in recommender systems, as users generally prefer a small number of high-confidence recommendations rather than a large set of mediocre options.

Model Interpretation and Limitations

- **SVD (Singular Value Decomposition)** effectively captures latent user preferences, uncovering patterns that are not immediately obvious from raw ratings.
- **Genre-based fallback** helps address cold-start scenarios for new users or newly added movies.
- **Limitations:** The model still faces challenges with entirely new genres or niche content. While the hybrid approach improves coverage, cold-start issues are not fully eliminated.

Conclusions and Business Recommendations

- **SVD with hyper-parameter tuning** consistently outperforms baseline and item-item collaborative filtering models.
- **Hybrid strategy** enhances robustness, particularly for new users or recently added movies.
- **Top-N evaluation metrics** (Precision@5, Recall@5) indicate that recommended items are relevant and actionable.
- Visualizations reveal key patterns in user preferences, providing actionable insights for stakeholders.

Future Work:

- Integrate richer content features such as actors, directors, and user reviews to further enhance recommendation quality.
- Explore weighted hybrid approaches to balance collaborative and content-based recommendations for improved coverage.
- Develop infrastructure for **real-time recommendations**, enabling immediate, personalized suggestions in streaming or high-traffic environments.